

Term	Definition
JavaScript	JavaScript is a high-level, interpreted, object-oriented, and event-driven programming language used to create interactive and dynamic web applications.
Variable	A variable is a named container used to store data that can be accessed and modified during program execution.
Scope	Scope is the area of a program where a variable or function can be accessed.
Global Scope	Global scope is the scope where variables are accessible from anywhere in the program.
Local Scope	Local scope is the scope where variables are accessible only within the function or block in which they are declared.
Block Scope	Block scope limits the accessibility of variables declared with <code>let</code> and <code>const</code> to the block <code>{ }</code> in which they are defined.
Function Scope	Function scope means variables declared with <code>var</code> are accessible only within the function where they are declared.
Hoisting	Hoisting is JavaScript's behavior of moving variable and function declarations to the top of their scope before code execution.
Closure	A closure is a function that remembers and can access variables from its outer lexical scope even after the outer function has finished executing.
Currying	Currying is the process of converting a function with multiple arguments into a sequence of functions, each taking one argument.
Callback Function	A callback function is a function passed as an argument to another function and executed later.
Promise	A Promise is an object that represents the eventual completion or failure of an asynchronous operation.
Async Function	An async function is a function that automatically returns a Promise and allows asynchronous code to be written more clearly.

Await	The <code>await</code> keyword pauses the execution of an <code>async</code> function until a Promise is resolved or rejected.
Synchronous Programming	Synchronous programming executes code line by line, waiting for each operation to complete before moving to the next.
Asynchronous Programming	Asynchronous programming allows tasks to run independently without blocking the execution of other code.
Event Loop	The event loop is the JavaScript mechanism that continuously checks the call stack and callback queue to execute asynchronous tasks.
Call Stack	The call stack is a data structure that keeps track of function calls during program execution.
Execution Context	An execution context is the environment in which JavaScript code is executed and variables, functions, and this are managed.
Lexical Scope	Lexical scope means a function's scope is determined by where it is defined in the source code.
Deep Copy	A deep copy creates a completely independent copy of an object, including all nested objects and arrays.
Shallow Copy	A shallow copy creates a new object but copies only the top-level properties, while nested objects remain shared by reference.
Object	An object is a collection of key-value pairs used to store related data and functionality.
Array	An array is an ordered collection of values stored under a single variable.
Function	A function is a reusable block of code designed to perform a specific task.
Arrow Function	An arrow function is a shorter syntax for writing functions and does not have its own <code>this</code> binding.
Template Literal	A template literal is a string enclosed in backticks that supports interpolation and multiline text.
Destructuring	Destructuring is a syntax that extracts values from arrays or properties from objects into separate variables.

Spread Operator	The spread operator (...) expands the elements of an iterable or the properties of an object into individual values.
Rest Parameter	The rest parameter (...) collects multiple function arguments into a single array.
Optional Chaining	Optional chaining (?.) safely accesses nested object properties without causing an error if a value is null or undefined.
Nullish Coalescing Operator	The nullish coalescing operator (??) returns the right-hand value only when the left-hand value is null or undefined.
Short-Circuiting	Short-circuiting is the behavior of logical operators that stop evaluating expressions as soon as the result is determined.
Truthy Value	A truthy value is any value that evaluates to true in a Boolean context.
Falsy Value	A falsy value is a value that evaluates to false in a Boolean context.
Prototype	A prototype is an object from which other objects inherit properties and methods.
Prototype Inheritance	Prototype inheritance is the mechanism by which one object inherits properties and methods from another object's prototype.
Class	A class is a blueprint for creating objects with predefined properties and methods.
Constructor	A constructor is a special method used to initialize objects when they are created from a class.
Inheritance	Inheritance is the ability of one class or object to acquire the properties and methods of another.
Encapsulation	Encapsulation is the practice of bundling data and methods together while restricting direct access to internal details.
Polymorphism	Polymorphism is the ability of different objects to respond differently to the same method or operation.
Abstraction	Abstraction is the process of hiding implementation details while exposing only the essential functionality.

this Keyword	The this keyword refers to the object that is currently executing the function.
call()	call() invokes a function immediately with a specified this value and individual arguments.
apply()	apply() invokes a function immediately with a specified this value and arguments provided as an array.
bind()	bind() creates and returns a new function with a permanently bound this value without executing it immediately.
Strict Mode	Strict mode is a JavaScript feature that enforces stricter parsing and error handling to help write safer code.
JSON	JSON (JavaScript Object Notation) is a lightweight text format used for storing and exchanging data.
DOM	The Document Object Model (DOM) is a programming interface that represents an HTML document as a tree of objects.
BOM	The Browser Object Model (BOM) provides objects that allow JavaScript to interact with the browser environment.
Event	An event is an action or occurrence, such as a click or key press, that JavaScript can detect and respond to.
Event Bubbling	Event bubbling is the process where an event propagates from the target element up through its ancestor elements.
Event Capturing	Event capturing is the process where an event propagates from the root element down to the target element.
Debouncing	Debouncing is a technique that delays a function's execution until a specified period of inactivity has passed.
Throttling	Throttling is a technique that limits how frequently a function can execute within a specified time interval.
Memoization	Memoization is an optimization technique that stores the results of expensive function calls and reuses them when the same inputs occur again.

IIFE (Immediately Invoked Function Expression)	An IIFE is a function that executes immediately after it is defined.
Module	A module is a reusable JavaScript file that exports and imports code to organize applications into separate components.
Garbage Collection	Garbage collection is the automatic process of freeing memory occupied by objects that are no longer reachable in a program.
Freeze	<code>Object.freeze()</code> prevents adding, deleting, or modifying properties of an object.
Seal	<code>Object.seal()</code> prevents adding or deleting properties but allows modification of existing properties.
Prevent Extensions	<code>Object.preventExtensions()</code> prevents new properties from being added to an object while allowing modification and deletion of existing properties.
Deep Freeze	Deep freeze recursively freezes an object and all of its nested objects, making the entire structure immutable.